

Proyecto StreamFlix

Módulo 1: "The Raw Parse" - RDDs y Datos No Estructurados

- Pregunta Reflexiva (Anti-IA):
- ¿Por qué usarías ``reduceByKey`` en lugar de ``groupByKey`` si tuviéramos millones de líneas de logs?
- ¿Qué ocurriría en memoria si usamos `groupByKey` con 200M de registros donde una clave concentra el 80% de los datos?
- Justifica tu respuesta con el concepto de "Shuffling".

Antes de explicar porque usaría `.reduceByKey()` antes que `.GroupByKey()`, voy a explicar el concepto de **Shuffling**. Es el proceso de Spark en el que se mueven datos por red de un ordenador a otro para poder agruparlos en este ordenador que recibe.

Usaría `.reduceByKey()` antes que `.GroupByKey()`, ya que este último realiza un **shuffling** masivo enviando muchos registros sin tratar por la red. En el caso de los "200M de registros donde una clave concentra el 80% de los datos", podemos decir que saturaría la RAM al enviar 160M en un solo nodo, provocando un error **Out Of Memory (OOM)**. Por eso, usar `.reduceByKey()` es una buena práctica, ya que realiza una pre-agregación o suma antes del **shuffle**, enviando solo un número para ser tratado por el nodo final y reduciendo el tráfico de red.

Validación manual.

```
Líneas descartadas: 792658
```

```
pedrocaballero@L2204028:~/Spark-Scala/StreamFlix$ cat src/main/resources/output/error_counts/part-  
502,82292  
503,82673  
500,82648
```

Módulo 2: Estandarización con DataFrames y Schemas

Validación manual.

- Pregunta: "Al definir el campo `subscription_price`, ¿por qué en un entorno bancario/facturación real deberías usar `DecimalType` en lugar de `DoubleType`? Muestra un ejemplo de código donde `DoubleType` pierda precisión en una suma."
- El alumno debe mostrar el esquema final (`printSchema()`) y 5 filas limpias.

En un entorno **bancario/facturación**, usaría **DecimalType** en vez de **DoubleType**, porque este último usa una representación binaria donde ciertos decimales no tienen representación exacta y esto provoca imprecisión. En cambio, **DecimalType** trabaja en base 10 y es exacto para cualquier número con hasta 2 decimales.

```
println("\n--- Validacion manual: DoubleType vs DecimalType ---")
//DoubleType:
val preciosDouble = Seq(14.99, 9.99, 4.99)
val sumaDouble    = preciosDouble.sum

println("=== DOUBLE ===")
println(f"14.99 + 9.99 + 4.99 = $sumaDouble%.17f")
println("\n--- resultado final limpio ---")

//DecimalType:
val preciosBigDecimal = Seq(
  BigDecimal("14.99"),
  BigDecimal("9.99"),
  BigDecimal("4.99")
)
val sumaBigDecimal = preciosBigDecimal.sum

println("\n=== BIGDECIMAL (equivalente a DecimalType en Spark) ===")
println(f"14.99 + 9.99 + 4.99 = $sumaBigDecimal")
```

```
--- Validacion manual: DoubleType vs DecimalType ---
=== DOUBLE ===
14.99 + 9.99 + 4.99 = 29.970000000000000000

--- resultado final limpio ---

=== BIGDECIMAL (equivalente a DecimalType en Spark) ===
14.99 + 9.99 + 4.99 = 29.97
```

Esquema final (printSchema()) y 5 filas limpias.

```
--- Esquema final y 5 filas limpias ---
root
|-- id: long (nullable = true)
|-- title: string (nullable = true)
|-- genres: string (nullable = false)
|-- subscription_price: double (nullable = true)
|-- release_date: string (nullable = true)
|-- country: string (nullable = true)

+---+-----+-----+-----+-----+-----+
| id|          title|          genres|subscription_price|release_date|country|
+---+-----+-----+-----+-----+-----+
| 1|Compatible system...|    Action|Horror|          9.78| 1992-02-28|  DE| |
| 2|Organic tertiary ...|    Action|Romance|          8.64| 2012-06-20|  UK|
| 3|Phased intermedia...|Sci-Fi|Drama|Thri...|          5.27| 2006-05-24|  ES|
| 4|Pre-emptive impac...|  Drama|Action|Comedy|         12.32| 1982-04-13|  UK|
| 5|Multi-lateral cli...|Comedy|Drama|Thri...|         16.37|invalid-date|  US|
```

Módulo 3: Enriquecimiento de datos (Joins & Broadcast)

Validación manual.

- El alumno debe explicar el "Physical Plan" (enrichedDF.explain()) y señalar dónde aparece BroadcastHashJoin.

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- BroadcastHashJoin [movie_id#15L], [id#39L], Inner, BuildRight, false
   :- Project [_1#5L AS user_id#14L, _2#6L AS movie_id#15L, _3#7L AS duration_watched#16L, _4#8 AS timestamp#17]
      :- SerializeFromObject [knownnotnull(assertnotnull(input[0, scala.Tuple4, true]))._1 AS _1#5L, knownnotnull(assertnotnull(input[0, scala.Tuple4, true]))._2 AS _2#6L, knownnotnull(assertnotnull(input[0, scala.Tuple4, true]))._3 AS _3#7L, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, scala.Tuple4, true]))._4, true, false, true) AS _4#8]
         :- Scan[obj#4]
         :- BroadcastExchange HashedRelationBroadcastMode(List(input[0, bigint, false]),false), [id=#288]
         :- Filter isnotnull(id#39L)
         :- FileScan csv [id#39L,title#40,genres#41,subscription_price#42,release_date#43,country#44] Batched: false, DataFilters: [isnotnull(id#39L)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/pedro/caballero/Spark-Scala/StreamFlix/src/main/resources/data...], PartitionFilters: [], PushedFilters: [IsNotNull(id)], ReadSchema: struct<id:bigint,title:string,genres:string,subscription_price:string,release_date:string,country...
```

Physical Plan, nos muestra los algoritmos exactos que Spark elige utilizar en el Clúster para la lectura de los datos, como los va a filtrar y como une las tablas, estas son las transformaciones que hicimos en el módulo 3 (.withColumn, .join, .groupBy), que son lazy evaluation y no se ejecutan inmediatamente.

BroadcastHashJoin, es un algoritmo de unión **join** muy rápido y eficiente dentro de spark, tiene una condición para su funcionamiento, una de las dos tablas que unimos debe ser más pequeña que la otra.

Sin **BroadcastHashJoin**: Spark mueve partes de las dos tablas a través de la red para encontrar coincidencias provocando un **Shuffle**.

Con **BroadcastHashJoin**: Spark detecta la tabla más pequeña, hace una copia de ella y se la envía a todos los nodos. De esta manera estos nodos de forma local tienen toda la información para cruzar los datos.

En la tercer línea de la captura de nuestro resultado podemos ver:

+ BroadcastHashJoin [movie_id#15L], [id#39L], Inner, BuildRight, false

Donde podemos ver y analizar que, efectivamente está utilizando **Broadcast** para cruzar las tablas de forma eficaz, nos indica que está haciendo un “**inner join**” clásico, y que está utilizando la columna **movie_id** de una tabla y la columna **id** de la otra para encontrar las coincidencias. Por otra parte **BuildRight** nos aclara que Spark decide usar la tabla de la derecha (tabla pequeña) para construir un índice en memoria. Para finalizar, el término **false**, nos indica que no utilizara nulos en la tabla nueva, aunque no tiene mucho sentido utilizar este término, ya que cruzamos las tablas con coincidencias de ambas tablas.

- **¿Qué ocurriría si el 60% de los logs pertenecen a la misma película?**

Esto va a provocar un problema llamado **Data Skew**, con el rendimiento de los datos que se distribuyen de forma desigual entre las particiones, haciendo que **Spark** sufra al estar obligado a mandar todos los datos que tengan la misma clave al mismo servidor, cargando un solo nodo en este caso con el 60% de la información.

La **consecuencia** de esto, es que rompe las ventajas del procesamiento paralelo. Mientras la mayoría de los nodos/servidores terminan rápido su proceso, el servidor cargado con el 60% de los datos, se convierte en un cuello de botella (esto en **Spark** se llama **Stragler Task**) provocando que el tiempo de procesamiento sea mayor y peor aún que ese nodo/servidor se quede sin memoria RAM, haciendo que el programa falle (**OutOfMemory**).

- **Resultado esperado: Una tabla ordenada por horas totales de visualización.**

```
--- Top géneros por horas ---
+-----+-----+
| genres|total_hours|
+-----+-----+
| Horror| 196772991|
| Action| 195207395|
| Drama| 58346040|
| Sci-Fi| 57046144|
| Romance| 56611662|
| Thriller| 56255090|
| Comedy| 54768148|
+-----+-----+
```

Módulo 4: Análisis de Comportamiento (Window Functions)

Validación manual:

- **Análisis:** El dataset contiene un usuario **User:999** que tiene tiempos negativos (errores de reloj). El alumno debe identificarlo y explicar con su lógica cómo manejó ese caso (¿lo filtró antes o rompió el cálculo?).

El **user:999** tiene `diff_minutes` negativos por errores de reloj. La condición `'diff_minutes > 0'` los descarta automáticamente. **is_binge = false** para todos los tiempos negativos.

```
--- Validación Manual ---
+-----+-----+-----+-----+
|user_id|timestamp|diff_minutes|is_binge|
+-----+-----+-----+-----+
| 999|2026-02-24 10:09:06|-6.7|false|
| 999|2026-02-24 10:09:36|-128.5|false|
| 999|2026-02-24 10:09:57|-136.65|false|
| 999|2026-02-24 10:10:13|-10.733333333333333|false|
| 999|2026-02-24 10:12:44|-177.48333333333332|false|
+-----+-----+-----+-----+
only showing top 5 rows
```

```
//User:999 que tiene tiempos negativos
bingeDF
  .filter(col("user_id") === 999 && col("diff_minutes") < 0)
  .select("user_id", "timestamp", "diff_minutes", "is_binge")
  .show(5)
```

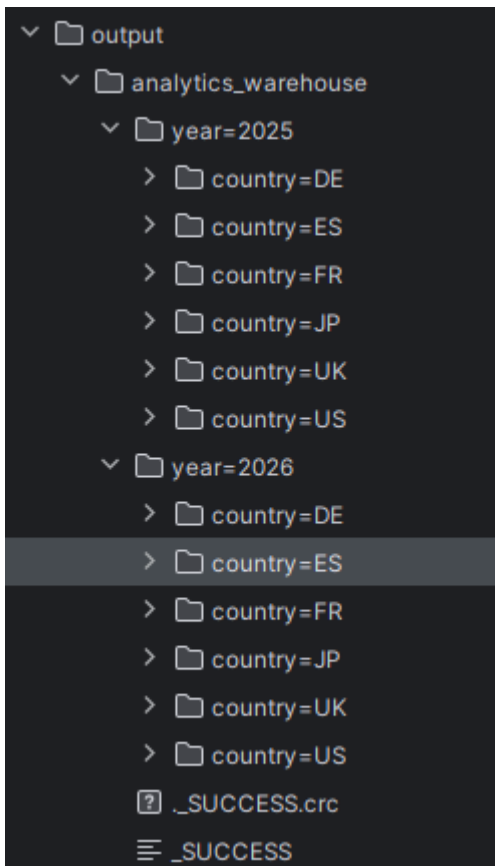
- **Entregable:** ID de los 3 usuarios más adictos.

```
--- Top 3 usuarios más adictos ---
+-----+-----+
|user_id|binge_count|
+-----+-----+
| 999|25|
| 3003|5|
| 5005|5|
+-----+-----+
```

Módulo 5: Particionamiento y Escritura Eficiente

Validación Manual:

- El alumno debe mostrar una captura de pantalla de su explorador de archivos mostrando la estructura de carpetas creada: Plaintext
/output/analytics_warehouse/year=2023/country=ES/part-0000.parquet
/output/analytics_warehouse/year=2023/country=US/part-0000.parquet



- **Pregunta: "¿Por qué NO debemos particionar por user_id en este caso? (Pista: Small files problem)".**

No debemos particionar por **user_id** porque causaría el "Small Files Problem". Si tenemos 1 millón de usuarios, Spark intentará crear 1 millón de carpetas (/user_id=1/, /user_id=2/, etc.), cada una conteniendo un archivo Parquet diminuto (quizás de unos pocos Kilobytes). **Hadoop** y **Spark** gestionan los datos mediante un nodo maestro/NameNode que debe guardar la metadata de cada archivo en su memoria RAM. Tener millones de archivos pequeños colapsará la memoria del clúster. Además, el tiempo que tarda Spark en abrir y cerrar millones de archivos pequeños arruina por completo el rendimiento, volviendo la lectura muy lenta. Las particiones deben hacerse siempre sobre columnas de baja cardinalidad (pocos valores distintos), como el año, mes, o país, garantizando que cada archivo resultante pese idealmente entre 128 MB y 1 GB.

